Universidad de Las Palmas de Gran Canaria

Grado en Ciencia e Ingeniería de Datos

Big Data - Academic Year 2026/2027

Performance Comparison of Basic Matrix Multiplication in Python, Java and C

Individual Assignment 1

Sara Carrizosa Tomé

GitHub Repository

`https://github.com/saracarri/matrix_multiplication`

October 2026

Abstract

This study compares the performance of basic matrix multiplication implemented in Python, Java and C. The same triple-loop algorithm was used in all three languages, with square matrices of increasing sizes from 10 to 300. Execution time and memory consumption were measured under the same hardware conditions, using shared input matrices to maintain a consistent comparison.

The results show that execution time increases considerably as matrix size grows, following a trend consistent with the expected cubic complexity. C achieved the lowest execution times in the tested configurations, followed by Java and Python. Differences were also observed in process memory usage, although these measurements include the overhead of each language's execution environment.

The experiment illustrates how implementation and runtime characteristics can affect the practical performance of an algorithm with the same theoretical complexity.

1 Introduction

Matrix multiplication is widely used in scientific computing, data processing and machine learning. As matrix dimensions increase, the number of operations required also grows, which makes its performance an interesting subject to investigate.

Although the theoretical complexity of an algorithm does not depend on the programming language, its practical performance can vary considerably. Factors such as compilation, memory management and data representation can affect both execution time and memory usage.

This experiment compares a basic matrix multiplication algorithm implemented in Python, Java and C. The same triple-loop approach was used in all three cases, without relying on optimised numerical libraries. The aim is to examine how matrix size affects execution time and memory consumption under the same hardware conditions.

The main research question is: How does matrix size affect the execution time and memory usage of basic matrix multiplication in Python, Java and C?

The initial hypothesis is that execution time will increase approximately according to the cubic complexity of the algorithm. However, differences between the three implementations are expected due to their execution environments and memory management. Memory requirements are also expected to increase as matrix dimensions grow.

2 Background

The standard algorithm for multiplying two square matrices uses three nested loops. Given two matrices A and B of size $n \times n$, each element of the resulting matrix C is calculated as:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj} \tag{1}$$

The first two loops select the position (i, j) of the output matrix, while the third loop calculates the sum of the products needed for that position.

Since the algorithm performs n iterations at each of the three levels, its time complexity is $O(n^3)$. Therefore, doubling the matrix dimension is expected to increase the amount of arithmetic work by a factor of eight.

The storage required for three dense matrices grows quadratically with their dimensions, giving a space complexity of $O(n^2)$. However, the actual memory consumed by a program also depends on how each language stores its data and manages its execution environment.

3 Methodology

3.1 Implementations

The matrix multiplication algorithm was implemented separately in Python, Java and C. All three versions follow the same i-j-k loop order, where the first two loops select an element of the output matrix and the third calculates its value.

The implementations use integer arithmetic and store the matrices in dense structures. Python uses nested lists, Java uses two-dimensional integer arrays, and C uses contiguous memory allocated dynamically.

To make the timing comparison more consistent, the output matrix was allocated before starting the measurements and reused between repetitions. Each multiplication resets its elements to zero before calculating the new values.

No external numerical libraries were used to perform the multiplication. However, the languages differ in their execution environments and data representations. Python uses arbitrary-precision integers, while Java uses 32-bit integers and C uses the int type, which occupies four bytes in the tested environment.

3.2 Input Data

The experiments used square matrices with dimensions 10, 50, 100, 200 and 300. These sizes were selected to examine the behaviour of the implementations as the computational workload increased, while keeping execution times manageable.

The input matrices were generated in Python using a fixed random seed. Seeds 42 and 43 were used for matrices A and B, respectively, with integer values ranging from 0 to 9.

The generated matrices were saved as CSV files and reused by all three implementations. This ensured that each language performed the multiplication using exactly the same input values.

For correctness verification, reference results were calculated separately and saved for each matrix size. The implementations were checked against these results before analysing their performance.

3.3 Experimental Environment

All experiments were performed on the same computer, equipped with an Intel Core Ultra 7 155H processor, 16 physical cores, 22 logical processors and approximately 32 GB of RAM. The operating system was Windows 11 Home.

The implementations were executed using Python 3.12.6, OpenJDK Temurin 17.0.15 and GCC 16.2.0 through MSYS2 UCRT64.

The C implementation was compiled with the flags `-std=c11`, `-O2`, `-Wall` and `-Wextra` [3]. Java was executed using the standard JVM configuration, while Python was run using its standard interpreter.

All implementations were executed sequentially, without explicitly creating additional worker threads.

3.4 Measurement Procedure

Execution time was measured using a monotonic timer in each language. Python used `time.perf_counter()` [1], Java used `System.nanoTime()` [2], and C used `CLOCK_MONOTONIC`.

Input matrices were loaded before starting the timer. The measured interval included the multiplication and the initialisation of the output elements, but excluded input loading, memory allocation and correctness verification.

Each configuration was measured five times. Python and C performed one warm-up execution, while Java performed ten to reduce the influence of JVM startup and runtime optimisation. Warm-up executions were excluded from the reported measurements.

For matrices of size 10 in C, each measurement consisted of 1000 consecutive multiplications. The total elapsed time was divided by the batch size to obtain the average time per multiplication. Larger matrices were measured individually.

The median was used to represent execution time, while the interquartile range (IQR) was used to describe variability. All individual measurements were preserved in CSV files.

Memory usage was investigated through two approaches. The storage required for the matrix elements was estimated from their dimensions and numeric types. In addition, the resident set size (RSS) of each process was sampled using Python's `psutil` library [4].

These RSS measurements include the execution environment and other process allocations, rather than only the memory required by the matrices. Short-lived memory peaks may also be missed because the measurements were collected periodically.

For the five measured repetitions, the IQR was calculated as the difference between the fourth and second values after sorting the execution times. This method was applied consistently across the three languages.

Process memory was measured once for each language and matrix size. Therefore, the reported RSS values represent individual observations rather than averages or medians of repeated measurements.

Before testing larger matrices, a maximum execution time of 30 seconds per multiplication and a memory budget of 1 GiB were established. All five tested matrix sizes remained within these limits, with 300 being the largest size evaluated.

4 Results

4.1 Execution Time

Table 1 presents the median execution times obtained for each implementation, together with their interquartile ranges. All values are expressed in milliseconds.

Table 1: Median execution time and IQR (ms)

n	Python		Java		C	
	Median	IQR	Median	IQR	Median	IQR
10	0.0605	0.0006	0.0255	0.0005	0.000553	0.000060
50	7.8473	0.4748	0.3871	0.0098	0.0420	0.0500
100	65.6584	1.2969	2.3080	0.1157	0.4080	0.0556
200	567.9850	26.4637	12.3559	0.5729	3.4651	0.5788
300	1790.7171	43.4096	44.3686	0.8260	10.5714	1.7201

Execution time increased with matrix size in all three implementations. C recorded the lowest median times for every tested size, followed by Java and Python.

The differences became more noticeable for larger matrices. At n = 300, the median execution time was approximately 1791 ms in Python, 44 ms in Java and 11 ms in C.

The measured growth was not perfectly cubic in every case, and some configurations showed greater variability than others. These differences are examined further in the discussion.

Figure 1 shows the median execution times on a linear scale. Since Python takes considerably longer for larger matrices, the differences between Java and C are difficult to distinguish in this figure.

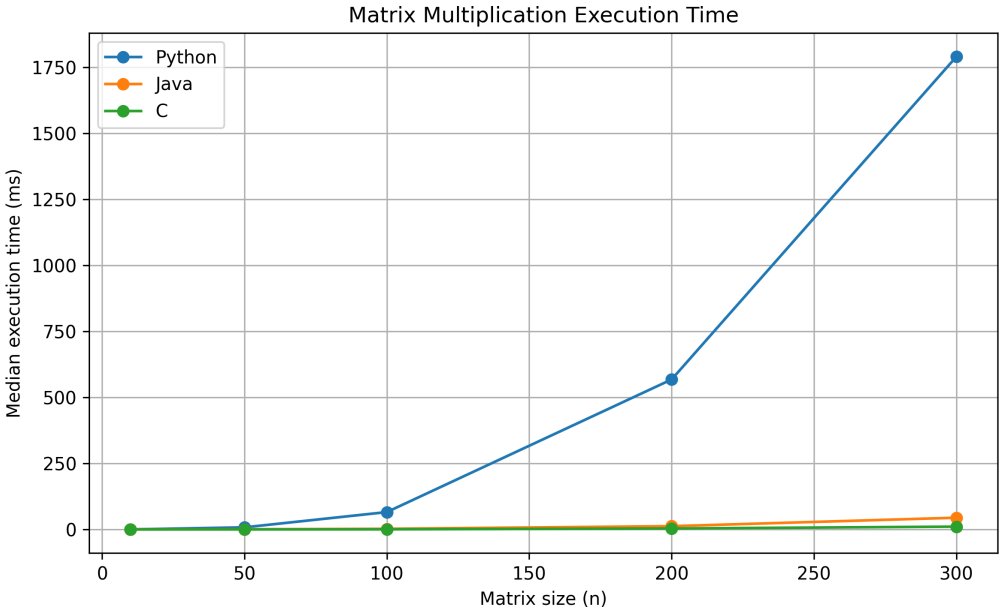


Figure 1: Median execution time for the three implementations.

Figure 2 presents the same results using a logarithmic vertical axis, allowing the three implementations to be compared more clearly.

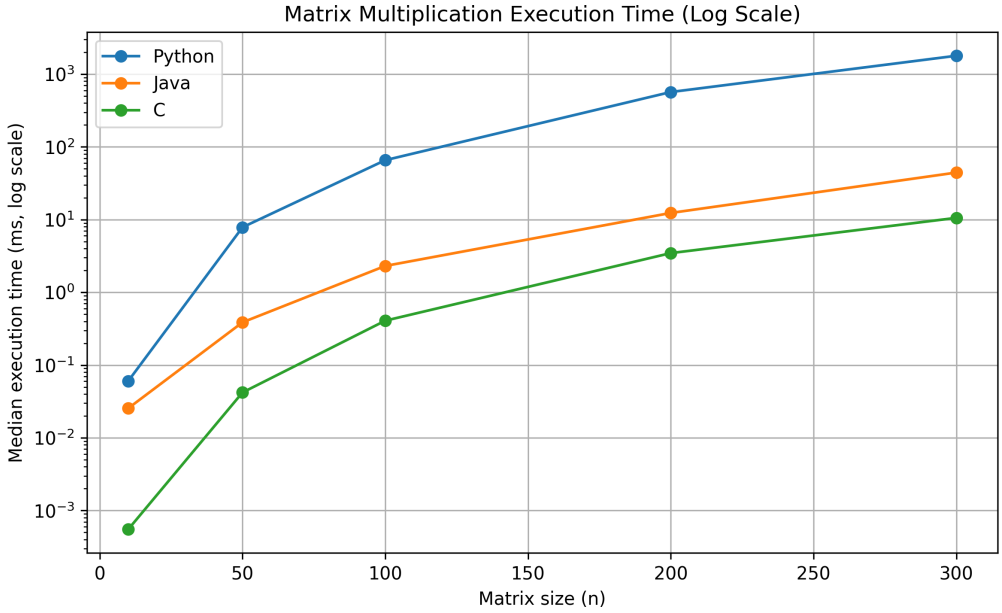


Figure 2: Median execution time using a logarithmic scale.

4.2 Memory Usage

Memory consumption was examined using the highest observed resident set size (RSS) for each program and matrix size. The measurements include the memory occupied by the execution environment, not only the matrices.

Table 2: Observed peak RSS (MiB)

n	Python	Java	C
10	13.7461	44.5469	5.5430
50	14.3438	47.3164	5.5859
100	16.0078	48.6875	5.5781
200	20.6523	59.5391	6.0000
300	28.1797	70.0469	6.5938

The observed memory usage generally increased as matrix size grew. Java showed the highest process memory consumption, while C showed the lowest values across the tested configurations.

At n = 300, the observed peak RSS was approximately 28.18 MiB for Python, 70.05 MiB for Java and 6.59 MiB for C. These differences also reflect the memory required by the respective execution environments.

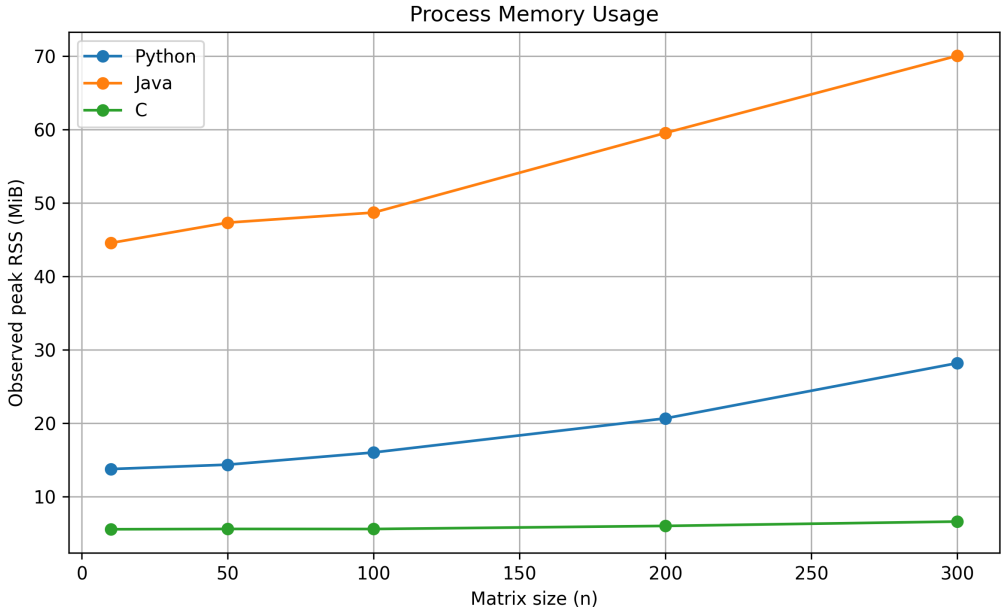


Figure 3: Observed peak process RSS for different matrix sizes.

5 Discussion

The results show that matrix size has a considerable effect on execution time in all three implementations. As expected, larger matrices require more operations, which is consistent with the cubic complexity of the algorithm. However, the increase in execution time was not identical across languages or matrix sizes.

For example, when the matrix dimension increased from 100 to 200, the median execution time increased by approximately 8.7 times in Python, 5.4 times in Java and 8.5 times in C. Although the theoretical amount of arithmetic work increases by a factor of eight, actual execution times are also affected by memory access, runtime behaviour and system activity.

Another clear difference is the performance of the three implementations. C achieved the lowest execution times throughout the experiment, followed by Java and Python. One possible explanation is that C was compiled into native machine code with optimisation enabled, while Java relies on JVM runtime optimisation and Python executes the loops through an interpreter.

Differences in data representation may also contribute to these results, since Python uses nested lists of integer objects rather than the more compact integer arrays used in Java and C.

Memory measurements followed a different pattern. Java showed the highest observed process memory usage, followed by Python and C. However, these values include the memory required by the JVM, Python interpreter and other process allocations. Therefore, the differences cannot be attributed exclusively to the matrices themselves. The estimated storage required for the matrix elements increased quadratically with matrix size, as expected.

Some limitations should also be considered. The experiments were performed on a single computer, using one implementation per language and a limited range of matrix sizes. Small differences in execution conditions may have influenced the measurements, particularly for the shortest operations. In C, a batch of 1000 multiplications was used for the smallest matrix size to reduce timing overhead, although this approach introduces a slightly different measurement procedure.

In addition, the observed peak RSS was obtained through periodic sampling, which may miss short-lived memory peaks. Only one memory measurement was collected for each configuration, so the variability of memory consumption could not be evaluated.

Overall, the experiment supports the expected relationship between matrix size and computational cost, while showing that implementations with the same theoretical complexity can have substantially different practical performance.

6 Conclusions

This experiment investigated how matrix size affects the execution time and memory usage of basic matrix multiplication in Python, Java and C.

The results showed that execution time increased considerably as matrix dimensions grew, following a trend consistent with the expected cubic complexity. Although all three implementations used the same algorithm and input matrices, their performance differed substantially. C recorded the lowest execution times in the tested configurations, followed by Java and Python.

Memory consumption also increased with matrix size, although the observed differences between languages were influenced by their execution environments. In particular, the process memory measurements included runtime overhead and could not be interpreted as the memory required by the matrices alone.

The findings are limited to the implementations, hardware and configurations used in this study. Further experiments with larger matrices, additional repetitions and different optimisation settings could provide a more complete understanding of their performance.

Use of Artificial Intelligence

Artificial intelligence tools, particularly ChatGPT (OpenAI), were used during the development of this assignment to support experimental planning, code implementation and debugging, and the organisation and language revision of the report.

AI assistance was mainly used when preparing the benchmarking and validation scripts, interpreting the experimental results and drafting sections of the report. The implementations were executed and tested locally, and all reported measurements were obtained from the actual experiments rather than generated by AI.

The author remained responsible for checking the implementations, reviewing the results and interpreting the findings.

References

[1] Python Software Foundation, "time — Time access and conversions," Python 3.12 Documentation. Available: https://docs.python.org/3.12/library/time.html

[2] Oracle, "Class System," Java Platform, Standard Edition 17 API Specification. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/System.html

[3] Free Software Foundation, "Optimize Options," Using the GNU Compiler Collection. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

[4] G. Rodola, "psutil: Process and System Utilities for Python." Available: https://psutil.io/